11 · Risk map

Each locality on a map, coloured by the share of its citizens with an open flag right now.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The **Risk map** page draws an OpenStreetMap map with one circle per locality and a ranking beside it. Colour shows the risk level (always with a text label too); circle size grows with the number of citizens. Like reports, it shows totals only and merges localities with fewer than 3 citizens.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/phc.py          GET /phc/map (with the facility position), GET /phc/locality-risk
└── services/localities.py     locality_risk(), risk_level()
frontend/src/
├── pages/phc/RiskMap.tsx      the page (react-leaflet map + ranking)
└── services/phc.ts            getMap()
```

How the pieces connect

1. `GET /phc/map` returns the facility position and, per locality: citizens, flagged citizens, share, level and average position.
2. The page centres the map on the facility and draws a `CircleMarker` for each locality that has a position.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/services/localities.py` · risk_level

```
27  def risk_level(share: float) -> RiskLevel:
28      if share >= 0.3:
29          return "high"
30      if share >= 0.15:
31          return "elevated"
32      return "some" if share > 0 else "none"
```

What the code means

- The level from the share of citizens with an open flag: 30% or more is high, 15% or more elevated, above 0 some, otherwise none.

2. `backend/app/services/localities.py` · locality_risk

```
50  async def locality_risk(session: AsyncSession, worker: User) -> list[dict]:
51      """Localities ranked by the share of their citizens with at least one open flag."""
52      flagged_ids = (
53          select(ProcessedData.user_id)
54          .where(ProcessedData.is_anomaly.is_(True), ProcessedData.resolution.is_(None))
55          .distinct()
56          .scalar_subquery()
57      )
58      citizens = catchment_citizens(worker).subquery()
59      rows = await session.execute(
60          select(
61              citizens.c.area,
62              func.count().label("citizens"),
63              func.count().filter(citizens.c.id.in_(flagged_ids)).label("flagged"),
64              func.avg(citizens.c.geo_lat).label("lat"),
65              func.avg(citizens.c.geo_lng).label("lng"),
66          ).group_by(citizens.c.area)
```

```
    )
    merged: dict[str, dict[str, float]] = {}
    coords: dict[str, tuple[float | None, float | None]] = {}
    for area, n, flagged, lat, lng in rows.all():
        name = locality_of(area)
        bucket = merged.setdefault(name, {"citizens": 0, "flagged": 0})
        bucket["citizens"] += n
        bucket["flagged"] += flagged
        coords.setdefault(name, (lat, lng))

    items = []
    for name, values in suppress(merged).items():
        share = values["flagged"] / values["citizens"] if values["citizens"] else 0.0
        lat, lng = coords.get(name, (None, None)) if name not in (OTHER, UNSPECIFIED) else (None, None)
        items.append(
... (12 more lines in the file: building the result list and sorting follow)
```

What the code means

- `flagged_ids` is a subquery of citizens with at least one open flag; `.scalar_subquery()` lets it be used inside `.in_(...)`.
- `func.count().filter(citizens.c.id.in_(flagged_ids))` counts flagged citizens per area in the same query as the totals.
- `func.avg(citizens.c.geo_lat)` gives an approximate centre for the circle.
- `merged.setdefault(name, {...})` creates the entry the first time a name is seen; areas that differ only by spacing end up together.

3. `backend/app/api/routes/phc.py` · risk_map

```
@router.get("/map", response_model=MapOut)
async def risk_map(worker: User = Depends(require_catchment_worker), db: AsyncSession = Depends(get_session)
    ) -> MapOut:
    facility = await db.get(ValidPhcFacility, worker.phc_id)
    assert facility is not None
    return MapOut(
        facility=_facility_out(facility),
        facility_lat=facility.latitude,
        facility_lng=facility.longitude,
        items=[LocalityRisk(**i) for i in await locality_risk(db, worker)],
        min_cell=MIN_CELL,
    )
```

What the code means

- `assert facility is not None` documents an assumption: `require_catchment_worker` already guaranteed a linked facility.
- `LocalityRisk(**i)` turns each dictionary into the response model.

4. `frontend/src/pages/phc/RiskMap.tsx` · LEVEL

```
const LEVEL: Record<LocalityRisk["level"], { label: string; color: string; text: string }> = {
  high: { label: "High", color: "rgb(var(--hn-sev-critical))", text: "text-sev-critical" },
  elevated: { label: "Elevated", color: "rgb(var(--hn-sev-high))", text: "text-sev-high" },
  some: { label: "Some", color: "rgb(var(--hn-sev-moderate))", text: "text-sev-moderate" },
  none: { label: "None", color: "rgb(var(--hn-success))", text: "text-success" },
};
```

What the code means

- A lookup from level to label and colour. `Record<LocalityRisk["level"], ...>` makes TypeScript check every level has an entry.

5. `frontend/src/pages/phc/RiskMap.tsx` · the map

```
33                <MapContainer center={centre} zoom={14} scrollWheelZoom={false} style={{ height: 420 }} aria-l
                      abel="Map of locality risk">
34                  <TileLayer attribution="&copy; OpenStreetMap contributors" url="https://{s}.tile.openstreetm
                        ap.org/{z}/{x}/{y}.png" />
35                  {placed.map((i) => (
36                    <CircleMarker
37                      key={i.locality}
38                      center={[i.lat!, i.lng!]}
39                      radius={10 + Math.sqrt(i.citizens) * 3}
40                      pathOptions={{ color: "rgb(var(--hn-surface))", weight: 2, fillColor: LEVEL[i.level].col
                            or, fillOpacity: 0.75 }}
41                    >
42                      <MapTooltip>
43                        {i.locality}: {LEVEL[i.level].label} · {i.flagged} of {i.citizens} flagged
44                      </MapTooltip>
45                    </CircleMarker>
46                  ))}
47                </MapContainer>
48              ) : (
49                <p className="p-5 text-sm text-ink-soft">No coordinates are recorded for this area; see the ra
                      nking.</p>
```

What the code means

- `<MapContainer center={centre} zoom={14}>` is the Leaflet map; `<TileLayer url=...>` loads the OpenStreetMap images.
- `placed.map((i) => <CircleMarker .../>)` draws a circle per locality. `radius={10 + Math.sqrt(i.citizens) * 3}`: square root keeps big localities from covering the map.
- `i.lat!` — the `!` tells TypeScript the value is not null here (the list was filtered to placed localities).
- Without any coordinates the page shows a sentence and the ranking instead of a map.

Syntax used in this feature

Code	What it means
`.scalar_subquery()`	A subquery usable as a value or inside IN (...).
`func.avg(col)`	SQL average.
`dict.setdefault(k, v)`	Get k, creating it with v if missing.
`x!`	TypeScript: "trust me, x is not null".